哈爾濱工程大學

第 07 讲 / 全 17 讲

C 语言 do-while 与循环控制详解

先做一次再问「还来吗」，还能随时喊停

do-while

先干一遍再问条件，循环体至少执行一次

break

猜中了、找到了就直接收工，跳出整个循环

continue

这趟剩下的不做了，跳回开头开始下一趟

循环综合

三种循环怎么选、穷举、累加、嵌套

哈尔滨工程大学 · C 语言程序设计　主讲教师：郎大鹏

本节目标

哈尔滨工程大学

1. 记住 do-while 的语法，尤其是 while（条件）后面那个不能省的分号
2. 说清 do-while 与 while 的唯一区别：条件首次不成立时执行几次
3. 判断什么时候该用 do-while ：至少要执行一次的场景
4. 用 break 提前结束整个循环，用 continue 跳过本次循环剩下的语句
5. 知道 break 只跳出一层，学会用标志变量让外层也停下来
6. 综合运用三种循环解决菜单、穷举、累加、嵌套等实际问题

01

do-while 循环

先做一次再问 · do while (条件); · 末尾分号 · 与 while 的区别 · 菜单场景

while 是进门先查证， do-while 是先玩一局再问「还玩吗」。

学完这六页，你能一眼看出某个循环「一次都不做行不行」，并选出正确的那一种。

哈爾濱工程大学

有些事必须先做一次再问

哈尔滨工程大学

第 06 讲的 while 是「先看条件，条件成立才干活」。可生活里有一类事，

必须先干一次，才知道要不要继续

先玩一局，出来再问「还玩吗」——这种事用 while 写别扭，do-while 才是原配。

while 的姿势

先看再干

门卫查证：先看证件，不合格直接不让进，一次都进不去

do-while 的姿势

先干再看

先玩一局、先显示菜单、先让你输一次，然后才问要不要继续

唯一的差别

至少 1 次

条件第一次就不成立时：while 一次不做，do-while 一定做一次

do-while 语法

先做后判

```
// 1. 先准备一个能进循环的变量
int n = 0;
// 2. do 后面就是要重复做的事
do {
    printf("输入一个数(0 结束): ");
    scanf("%d", &n);
} while (n != 0);
//              ^ 这个分号最容易忘
printf("循环结束");
// 顺序: 先做一次 → 再看条件 → 成立就回到 do
// 条件不成立时, 才第一次停下来
```

骨架长这样

do { 循环体 } while (条件);

do 打头、 while 收尾，循环体夹在中间。

末尾分号别丢

while (条件) 后面的分号是 do-while 的一部分，漏写编译直接报错。

至少执行一次

条件写在后面，第一次执行时还没轮到它说话。

do-while 与 while 的核心区别

本课最关键的一页

哈尔滨工程大学

把条件设成「一开始就不成立」： i 从 10 开始，条件是 i < 5 —— 两个循环的差别立刻现形

while ：先判后做

```
int i = 10;
while (i < 5)    //  进门先查条件
{
    printf("i=%d ", i);
    i++;
}
printf(" 结束 ");
//  屏幕输出：结束
//  循环体执行次数： 0  次
```

do-while ：先做后判

```
int i = 10;
do               //  先做一遍再说
{
    printf("i=%d ", i);
    i++;
} while (i < 5);    //  做完才查
printf(" 结束 ");
//  屏幕输出： i=10  结束
//  循环体执行次数： 1  次
```

什么时候用 do-while

判据只有一句： 问自己「这件事一次都不做行不行？」—— 不行，就用 do-while

```
// 菜单：不显示菜单，用户怎么知道按几？
int choice = 0;
do {
    printf("1. 查询   2. 充值   0. 退出： ");
    scanf("%d", &choice);
    if (choice == 1) printf(" 查询中 ");
    if (choice == 2) printf(" 充值中 ");
} while (choice != 0);
printf(" 再见，欢迎下次光临 ");
// 换成 while 写，菜单就得在循环外先抄一遍
```

菜单必须露面

不先把选项打出来，用户根本不知道该按几，一次都不显示没有意义。

输错了要重来

密码、成绩输错了，至少得让人重输一次。

反过来用 while

可能一次都不该做？那就用 while 。

用 do-while 重写猜数字

呼应第 06 讲

哈尔滨工程大学

回顾 猜数字：心里想好一个数，让用户反复猜，猜中为止 —— 「至少猜一次」

```
int target = 37, guess = 0;
do {
    printf("请猜一个数： ");
    scanf("%d", &guess);
} while (guess != target);
//    猜错就回到 do 那一行，再来一次
printf("猜中了！ ");
// 第 06 讲的 while 版：
// while (guess != target) { 提示 + 读入 }
// 两版效果一样， do-while 更贴人话
```

运行结果

```
请猜一个数： 50
请猜一个数： 25
请猜一个数： 37
猜中了！
```

为什么更顺

while 版要在循环外先抄一遍提示与读入，do-while 只写一遍。

while 版还得先给 guess 一个不等于答案的初值，do-while 免去这个坑。

do-while 的常见写法与缩进

```c
// 写法一：do 独占一行，最清楚（推荐）
do
{
    printf("再来一次？(1/0)：");
    scanf("%d", &again);
} while (again == 1);
// 写法二：do { 写在同一行，也很常见
do {
    printf("再来一次？(1/0)：");
    scanf("%d", &again);
} while (again == 1);
// 条件为真继续，为假退出；分号别丢
```

缩进：整体右移

循环体统一往右挪 4 个空格，do 与 while 上下齐平，一眼看出是一家的。

括号：一对要齐全

do 一个 {，while 前面一个 }，缺一个就编译不过。

循环体只有一句时

理论上可以省大括号，但新手一省就出错，建议永远写上。

02

break 与 continue

跳出整个循环 · 跳过本次 · 只跳一层 · 标志变量 · 死循环与菜单

循环不该只有「转满圈数」一种结局：找到答案可以提前收工，遇到不想处理的数据可以跳过。

学完这六页，你能给任何一个循环装上收工开关和跳过开关。

哈爾濱工程大学

break ：跳出整个循环

找到就收工

哈尔滨工程大学

场景 猜数字不限制次数：猜中了就没必要再问，直接结束循环

```
int target = 37, guess = 0;
while (1)            // 1 恒真：先跑起来
{
    printf(" 请猜一个数： ");
    scanf("%d", &guess);
    if (guess == target)
        break;       // 猜中：直接收工
}
printf(" 猜中了！ ");
```

运行结果

```
请猜一个数： 50
请猜一个数： 25
请猜一个数： 37
猜中了！
```

break 三句话

一执行，整个循环立刻结束，转去执行循环后面的语句。

break 跳的是循环，不是 if ；它只在循环（和 switch ）里才有意义。

continue ：跳过本次循环

场景 只处理偶数：遇到奇数就「这趟不干了」，直接开始下一趟

```
int i, sum = 0;
for (i = 1; i <= 10; i++)
{
    if (i % 2 != 0)  //  奇数
        continue;    //  跳回上面， i++ 照做
    sum = sum + i;   //  只有偶数走到这里
}
printf(" 偶数和  = %d", sum);
//  输出: 偶数和  = 30
//  如果把  continue  换成  break , 和就是  0
```

循环没有结束

continue 只是「这趟剩下的不做了」，下一趟照常开始。

跳到哪里

跳到循环体末尾： continue 下面的语句这次全部跳过。

for 里放心用

for 里 i++ 照做， while 里要当心

break 与 continue 对比

跳出 vs 跳过

哈尔滨工程大学

一个字之差，命运完全不同：一个让循环终止，一个让循环继续下一轮。

对比项	break（收工）	continue（跳过）
干什么	直接跳出整个循环，后面不转了	跳过本次剩下的语句，下趟照转
之后去哪	循环彻底结束，执行循环外的语句	回到循环条件判断，开始下一轮
常见场合	猜中了、找到了、够了	奇数跳过、负数跳过、不合法跳过

```
循环体 → break → 循环外的语句
```

```
循环体 → continue → 条件 → 下一轮
```

break 只跳出一层

嵌套循环的坑

哈尔滨工程大学

注意 两层循环里写 break ，它只结束自己所在的那一层，外层一点没少转

```
int i, j;
for (i = 1; i <= 3; i++)   //  外层
{
    for (j = 1; j <= 3; j++)  //  内层
    {
        if (j == 2)
            break;        //  只跳出内层
        printf("(%d,%d) ", i, j);
    }
}
//  输出: (1,1) (2,1) (3,1)
```

只认识自己那层

break 一次只能结束一层循环，从内往外数，它只管第一层。

看输出就知道

外层 i 的 1、2、3 全都出现了，说明外层一次没少。

想一次跳两层

C 语言不能一次跳两层，靠标志变量

用标志变量控制循环

立旗通知外层

哈爾濱工程大學

思路 内层找到目标就「立旗」，外层看一眼旗再决定停不停

```
int i, j, found = 0;  // 旗子: 0 还没找到
for (i = 1; i <= 3; i++)
{
    for (j = 1; j <= 3; j++)
        if (i * j == 6)
        {
            found = 1;  // 立旗
            break;      // 跳出内层
        }
    if (found == 1) break;  // 外层也停
}
```

第一步：立旗

目标达成时把标志变量置 1：found = 1，同时 break 跳出内层。

第二步：外层看旗

内层跑完后查一次 found == 1，成立就 break，外层也停。

为什么好使

标志变量像通知栏，比乱跳清晰得多

死循环与菜单模式

写法 while (1) 故意写成一个永远为真的循环，出口交给循环体里的 break

```
int choice = 0;
while (1)              //  故意写成死循环
{
    printf("1.开始   0.退出: ");
    scanf("%d", &choice);
    if (choice == 0)
        break;         //  唯一的出口
    printf("你选了 %d", choice);
}
printf("已退出程序");
```

死循环不可怕

只要里面一定有能执行到的 break ，它就是个可控的循环。

出口必须写

漏了 break 或者 break 的条件永远不满足，程序就真出不来了。

与 do-while 对比

do-while 用条件退出， while(1) 用 break

03

循环综合

三种循环怎么选 · 循环嵌套 · 穷举法 · 迭代累加 · 差一错误

会写循环不等于会选循环：这一部分的六页，把三种循环摆在一起比一比。

再补上穷举、累加、嵌套这些用循环解决问题的经典套路。

哈爾濱工程大學

三种循环怎么选

一张表定终身

循环	语法骨架	什么时候用	执行次数
for 三要素挤一行	`for (i = 0; i < n; i++)`	次数已知，数着走	0 次或多次
while 条件写在前面	`while ( 条件 ) {  循环体  }`	次数未知，先看再做	0 次或多次
do-while 条件写在后面	`do {  循环体  } while ( 条件 );`	至少要执行一次	1 次或多次

循环嵌套


```c
int i, j;
for (i = 1; i <= 3; i++)   //  外层：走一行
{
    for (j = 1; j <= 4; j++)  //  内层：走一圈
    {
        printf("(%d,%d) ", i, j);
    }
    printf("%c", 10);      //  换行
}
// (1,1)(1,2)(1,3)(1,4)
// (2,1)(2,2)(2,3)(2,4)
```

外层走一步

内层走一整圈。外层 i 每加 1 ，内层 j 就从 1 走到 4 。

变量别重名

外层用 i ，内层用 j ，写成同一个名字就乱套了。

总次数相乘

外层 3 次，内层 4 次，一共转 12 次。

穷举法：一个一个试

```
// 鸡兔同笼：头 35 个，脚 94 只
int head = 35, foot = 94;
int x, y;                    // x 鸡，y 兔
for (x = 0; x <= head; x++)  // 一个个试
{
    y = head - x;            // 兔数跟着定下来
    if (x * 2 + y * 4 == foot)
        printf("鸡%d 兔%d", x, y);
}
// 输出：鸡23 兔12
// 试完 36 种可能，条件成立的就是答案
```

穷举三件套

先定范围，再逐个试，最后用条件把答案筛出来。

范围要定准

鸡最多 35 只、最少 0 只，多了少了都可能漏掉答案。

验算别省

23 * 2 + 12 * 4 = 94，脚数对上了。

迭代与累加


```c
int i, x, sum = 0, cnt = 0, max = 0;
for (i = 1; i <= 5; i++)   // 读 5 个数
{
    scanf("%d", &x);
    sum = sum + x;      // 累加器
    cnt = cnt + 1;      // 计数器
    if (x > max)        // 打擂台
        max = x;
}
printf("和=%d 个数=%d 最大=%d", sum, cnt, max);
// sum 初值必须是 0，cnt 也是
```

累加器 sum

先清零，再一个一个往上加，忘了清零结果就是垃圾值。

计数器 cnt

每来一个数据就加 1，求平均时它就是分母。

打擂台 max

初值要比可能的数都小，来一个比一个，大的留下。

循环里的差一错误

n 还是 n-1

哈尔滨工程大学

差一错误 多算一个或少算一个，程序照样能跑，只是结果永远差那么一点点

常见错法

```
int i;
for (i = 1; i < 10; i++)  // 写成 < 了
{
    printf("%d ", i);
}
// 想输出 1~10，实际到 9 就停
for (i = 0; i <= 10; i++)  // 又写成 <=
// 想转 10 圈，实际转了 11 圈
```

正确写法

```
int i;
for (i = 1; i <= 10; i++)  // 含头含尾用 <=
{
    printf("%d ", i);
}
// 输出：1 2 3 ... 9 10，正好十个
for (i = 0; i < 10; i++)   // 从 0 起用 <
// 也是十圈：0 1 2 ... 9
// 口诀：数个数用 <=，从 0 起用 <
```

循环程序设计小结

1

循环变量初值

从几开始数？数个数从 1 起，数次数从 0 起。

```
i = 0;    i = 1;
```

2

继续条件

什么情况下接着转？条件写在括号里，成立才继续。

```
i < n;    i <= n;
```

3

更新语句

每轮怎么变？加 1、加 2 还是减 1，别忘写。

```
i++;    i += 2;
```

动手写之前，先答出这三问

一问：从哪里开始？二问：什么时候停？三问：每轮怎么变？三问答不出，就别急着敲代码。

写完还要回头验两次：第一次进来对不对（有没有差一），最后一次出去对不对（有没有死循环）。

break 与 continue 是加分项：能用它们让循环提前收工或跳过一趟，代码就更像人话。

例 1 · do-while 菜单

哈尔滨工程大学

例 1 反复显示菜单让用户选择，输入 0 时退出程序。

```
int main(void)
{
    int choice = 0;
    do {
        printf("1. 查询  2. 充值  0. 退出: ");
        scanf("%d", &choice);
        if (choice == 1) printf(" 查询中 ");
        if (choice == 2) printf(" 充值中 ");
    } while (choice != 0);    // 输入 0 就结束
    printf(" 再见，欢迎下次光临 ");
}
```

运行结果

```
1. 查询  2. 充值  0. 退出: 1
查询中
1. 查询  2. 充值  0. 退出: 0
再见，欢迎下次光临
```

要点

do-while 保证菜单至少出现一次，用户才知道该按几。

退出条件写在末尾：choice != 0 ，输入 0 时循环结束。

例 2 · 猜数字（ break 版）

例 2 猜数字升级版：至少猜一次，猜中立刻结束，并统计猜了几次。

```c
int main(void)
{
    int t = 37, g = 0, cnt = 0;
    do {
        printf(" 请猜一个数： ");
        scanf("%d", &g);
        cnt = cnt + 1;
        if (g == t) break;  // 猜中：收工
        printf(" 猜错了，再试 ");
    } while (1);   // 恒真：出口是 break
    printf(" 猜中！用了 %d 次 ", cnt);
}
```

运行结果

```
请猜一个数： 50
猜错了，再试
请猜一个数： 37
猜中！用了 2 次
```

要点

do-while 负责「至少猜一次」，break 负责「猜中就走」。

cnt 放在读入之后加 1，猜错猜中都算一次。

例 3 · continue 求偶数和

例 3 求 1 到 100 之间所有偶数的和，遇到奇数就用 continue 跳过。

```c
int main(void)
{
    int i, sum = 0;
    for (i = 1; i <= 100; i++)
    {
        if (i % 2 != 0)
            continue;   // 奇数：跳过下面
        sum = sum + i;  // 偶数：才累加
    }
    printf("1~100 偶数和 = %d", sum);
}
```

运行结果

```
1~100 偶数和 = 2550
```

2 + 4 + … + 100 ，一共 50 个偶数。

要点

continue 跳的是「本次剩下的语句」，循环照常继续。

for 里用 continue 是安全的：i++ 一定会执行，不会死循环。

例 4 · 反复输入求平均

例 4 反复输入成绩求平均分，输入 -1 表示结束（ -1 是哨兵值）。

```
int main(void)
{
    int s, cnt = 0; double sum = 0;
    do {
        printf(" 成绩 (-1  结束 )： ");
        scanf("%d", &s);
        if (s == -1) break;  // 哨兵：退出
        sum = sum + s;
        cnt = cnt + 1;
    } while (s != -1);
    if (cnt > 0) printf(" 平均  = %.1f", sum / cnt);
}
```

运行结果

```
成绩 (-1  结束 )： 90
成绩 (-1  结束 )： 80
成绩 (-1  结束 )： -1
平均  = 85.0
```

要点

读到 -1 立刻 break ，否则它会被当成成绩加进总分。

最后判断 cnt > 0 再相除，一个成绩都没输入时不会除零。

例 5 · 九九乘法表

例 5 打印九九乘法表：外层控制第几行，内层控制这一行的第几列。

```
int main(void)
{
    int i, j;
    for (i = 1; i <= 9; i++)   // 第 i 行
    {
        for (j = 1; j <= i; j++)  // 第 i 行有 i 个
            printf("%d*%d=%d ", j, i, i * j);
        printf("%c", 10);   // 一行结束换行
    }
}
// 内层写成 j <= i，才能打出三角形
```

运行结果

```
1*1=1
1*2=2  2*2=4
1*3=3  2*3=6  3*3=9
…… 一直打到第 9 行
```

要点

外层 i 走一步，内层 j 从 1 走到 i，走完换行。

把内层改成 j <= 9 就会打出方阵，试试看差别。

常见错误盘点

六个高频坑

哈尔滨工程大学

1 do-while 末尾忘写分号：} while (n != 0) 后面漏掉分号 —— 编译报错

2 while（条件）后面多写分号：循环体变成空语句 —— 条件恒真就死循环

3 while 里 continue 跳过了更新语句：i++ 被跳掉 —— 变量不变，死循环

4 break 只跳出一层：嵌套循环里外层照转 —— 该停的那层没停

5 循环变量没更新：忘了写 i++ 或写反方向 —— 永远出不来

6 边界差一：数 n 个数写成 i < n 少一个，从 0 起写成 i <= n 多一个

小结

循环这块拼图，今天终于拼齐了

do-while 先做一次再问条件，循环体至少执行一次； while 先看条件，可能一次不做。

break 让循环提前收工， continue 让本次剩下的语句跳过； break 只跳出一层，多层要靠标志变量。

写循环前先答三问：从哪开始、什么时候停、每轮怎么变。

下一讲：数组 —— 一个名字管住一整排数据

一个班 50 个同学的成绩，难道要定义 50 个变量？学完数组，一个名字加一个下标就能全装下。

哈爾濱工程大学